



INDIA.RUNS

Build what next India runs on

Team Name :Deepanshi Khandelwal

Team Leader Name :Deepanshi Khandelwal

Problem Statement :Intelligent Candidate Discovery & Ranking – Redrob AI

Solution Overview

- What is your proposed solution?

A CPU-only, zero-API multi-dimensional candidate ranking system that processes 100,000 profiles in 19 seconds. Combines TF-IDF semantic matching with 5 rule-based scoring dimensions — Skill Quality (28%), Experience Trajectory (25%), Semantic Relevance (22%), Availability Signals (15%), Platform Trust & Location (10%) — to produce a precise top-100 shortlist.

- What differentiates your approach from traditional candidate matching systems?
- **Reads JD meaning, not just words:** Encodes what the JD implies — consulting-only careers disqualified, LangChain-only penalised
- **Behavioral signals as first-class:** Platform inactivity and sub-5% recruiter response rate are hard scoring factors
- **Anti-gaming:** Honeypot detection drops ~80 fraudulent profiles before scoring begins
- **Precision over recall:** 10 perfect matches over 1,000 maybes — as explicitly stated in the JD

JD Understanding & Candidate Evaluation

Key Requirements from JD:

- 5–9 years applied ML production experience
- Required: Embeddings, FAISS, Pinecone, Weaviate, NDCG, MRR, Python
- Product company — NOT consulting (TCS/Infosys etc.)
- Shipped production systems with real users
- Location: Pune / Noida hybrid preferred
- Open to work + short notice period valued

Candidate Fit Beyond Keyword Matching:

- Skill proficiency levels: advanced/expert with usage months > 6 only count
- Career trajectory: product vs consulting, avg tenure patterns
- Platform activity: last_active_date, recruiter_response_rate, open_to_work flag
- GitHub score: open-source contributions as trust signal
- Endorsements + assessments: platform-verified skill scores carry extra weight

Ranking Methodology

Scoring Weights:

- Skill Quality: 28%
- Experience Trajectory: 25%
- Semantic TF-IDF: 22%
- Availability Signals: 15%
- Platform Trust: 5%
- Location Fit: 5%

Final Score Formula:

$Skill \times 0.28 + Exp \times 0.25 + Sem \times 0.22 + Avail \times 0.15 + Plat \times 0.05 + Loc \times 0.05$

Algorithm Steps:

1. Load 100K candidates (JSONL)
2. Honeypot detection → drop fakes
3. Quick rule-based pre-filter → Top 5K
4. TF-IDF cosine similarity to JD text
5. Full 6-dimension scoring per candidate
6. Weighted composite → sort → Top 100
7. Normalize 0.40–0.99 → write CSV/XLSX

Explainability & Data Validation

How are ranking decisions explained?

Every top-100 candidate receives an auto-generated recruiter-style reasoning string built from actual profile fields — current title, years of experience, location, matched skills, AI/ML roles in career history, notice period, last active date, and GitHub score. Nothing is invented. Example: *"7-year Senior ML Engineer, Noida. Relevant skills: Weaviate, Pinecone, Embeddings. 3 AI/ML roles including Senior ML Engineer at Zomato. Short notice period (15d). Active on platform recently. GitHub activity score 95/100."*

How do you prevent hallucinations or unsupported justifications?

Zero LLM calls — no API, no text generation. Every reasoning line is constructed by `build_reasoning()` which reads only actual candidate fields. If a field is missing it is silently skipped. The entire pipeline is deterministic and reproducible — running `rank.py` twice on the same input produces identical output.

How does your solution handle inconsistent, low-quality, or suspicious profiles?

`detect_honeypot()` runs before any scoring:

- 5+ "expert" skills with 0 months usage → dropped (impossible claim)
- 20+ expert skills total → dropped (impossible breadth)
- Self-reported YoE > 3.5× actual career history in months → dropped (inconsistent data)

This eliminates ~80 fraudulent profiles identified in the dataset before they can influence rankings.

End-to-End Workflow

Step 1 — JD Understanding

Job description parsed into dense `JD_TEXT` with required skills, location, YoE range, and disqualifiers.

Step 2 — Load Candidates

100,000 profiles loaded from `candidates.jsonl` — profile, career history, skills, and redrob_signals.

Step 3 — Honeypot Filter

`detect_honeypot()` removes ~80 fake profiles with impossible statistics before scoring begins.

Step 4 — Quick Pre-filter

Rule-based scoring narrows 100K → 5,000 candidates using title, YoE, skill count, and activity signals.

Step 5 — Semantic Scoring

TF-IDF bigram vectorizer fits on JD + 5K candidates. Cosine similarity gives each candidate a semantic score.

Step 6 — 5-Dimension Scoring

`score_skills()` · `score_experience()` · `score_availability()` · `score_platform_trust()` · `score_location()`

Step 7 — Weighted Combination

Final = Skill×0.28 + Exp×0.25 + Sem×0.22 + Avail×0.15 + Plat×0.05 + Loc×0.05

Step 8 — Output

Top 100 sorted by score → submission.xlsx with normalized scores and reasoning per candidate.

System Architecture

INPUT LAYER: candidates.jsonl (100K profiles, one per line)

FILTER LAYER: detect_honeypot() → quick_score() → Top 5,000 retained

SEMANTIC LAYER: build_candidate_text() → TfidfVectorizer (bigram, 60K, sublinear) → cosine_similarity vs JD_TEXT

SCORING LAYER: score_skills() | score_experience() | score_availability() | score_platform_trust() | score_location()

OUTPUT LAYER: submission.xlsx → candidate_id | rank | score | reasoning + Streamlit demo app

Results & Performance

Rank	Candidate ID	Role	Company	Location	Score
#1	CAND_0018499	Sr. ML Engineer	Zomato	Noida, UP	0.9900
#2	CAND_0086022	Sr. Applied Scientist	Sarvam AI	Kolkata, WB	0.8664
#3	CAND_0088025	Staff ML Engineer	Yellow.ai	Jaipur, RJ	0.8631
#4	CAND_0027691	NLP Engineer	Haptik	Pune, MH	0.7748
#5	CAND_0081846	Lead AI Engineer	Razorpay	Jaipur, RJ	0.7620

Technologies Used

- What technologies, frameworks, and tools were used and why were they selected for this solution?

Python 3.10 – Core language

scikit-learn TF-IDF – CPU-efficient semantic matching

NumPy – Array operations and cosine similarity

Pandas – Data handling and CSV/XLSX output

Streamlit – Live demo sandbox

Submission Assets

GitHub: <https://github.com/Deepanshik/Redrob-Ranker>

Sandbox: <https://redrob-ranker-ghdyvc8ojcsm28syn99gse.streamlit.app/>

Ranked Output: [output/submission.xlsx](#) — 100 rows, ranks 1-100

Reproduce: `python rank.py --candidates ./data/candidates.jsonl --out ./output/submission.csv`



INDIA.RUNS

Build what next India runs on

THANK YOU